Adding a Process-Subtree Monitoring and Enforcement System Call to the Linux Kernel

A Step-by-Step Implementation Guide

CS G623 – System Calls Assignment

October 12, 2026

Contents

1 The problem statement, restated precisely **2**

2 Background you need **2**
- 2.1 How a system call reaches your code (x86-64) . . . 2
- 2.2 Process relationships in the kernel . . . 2
- 2.3 Resource accounting already in the kernel . . . 3

3 Design **3**
- 3.1 Interface . . . 3
- 3.2 Where do the "limits" come from? . . . 4
- 3.3 What is the "appropriate action"? . . . 4

4 Environment setup **4**
- 4.1 Get the source . . . 4
- 4.2 Build tools (Arch Linux) . . . 4
- 4.3 Never test on your host . . . 4
- 4.4 Config . . . 5

5 Implementation steps **5**
- 5.1 Step 1: the user-space ABI header . . . 5
- 5.2 Step 2: the implementation file . . . 5
- 5.3 Step 3: register the syscall . . . 8
- 5.4 Step 4: build . . . 9

6 Testing **9**
- 6.1 User-space wrapper . . . 9
- 6.2 Test matrix . . . 9
- 6.3 Stress / debug . . . 10

7 Polish expected of a mainline-quality change **10**

8 Verification greps (names that drift between releases) **11**

9 Suggested work plan **11**

Read this first

This guide was written without a kernel tree at hand. The *ideas*, file locations and the overall structure are stable, but exact helper names, header locations and syscall numbers drift between releases. Every code block is therefore a *skeleton to be compiled and corrected against your tree*. Where a detail is worth double-checking I say so, and Section 8 lists greps to run.

1 The problem statement, restated precisely

The assignment asks for one new system call that, given a `pid`:

R1. reports the **resource utilisation** of that process (CPU time, memory, ...);

R2. **enforces limits** on it, i.e. takes an appropriate action if the process is over its limits;

R3. does so **only if** the target is the caller itself, or lies in the caller's **descendant tree** (children, grandchildren, great-grandchildren, ...);

R4. otherwise refuses.

Three things in this statement are under-specified and you must pin them down before writing code: *which* resources, *which* limits (and where they come from), and *what* "appropriate action" is. Section 3 proposes defaults; confirm them with your instructor.

2 Background you need

2.1 How a system call reaches your code (x86-64)

1. User space puts the syscall number in `rax`, arguments in `rdi, rsi, rdx, r10, r8, r9`, and executes `syscall`.
2. The CPU jumps to `entry_SYSCALL_64` (`arch/x86/entry/entry_64.S`), which saves registers and calls `do_syscall_64()` (`arch/x86/entry/syscall_64.c`).
3. That indexes `sys_call_table[nr]` and calls the handler. The table is *generated at build time* from `arch/x86/entry/syscalls/syscall_64.tbl`.
4. The handler is whatever you declared with `SYSCALL_DEFINEn(name, ...)`. The macro creates `__x64_sys_name`, wraps argument fetching/sign-extension, and registers the function for error injection and tracing.

So adding a syscall means: (a) write the function, (b) give it a number in the table, (c) declare its prototype, (d) wire it into the build, (e) expose a user-space ABI struct, (f) test.

2.2 Process relationships in the kernel

- Kernel unit of scheduling is `struct task_struct`; a *process* is a thread group. `task->group_leader` is the main thread; its PID is the TGID that user space calls "pid".
- `task->real_parent` is the task that `fork()`ed it (or its reaper, after reparenting). `task->parent` may differ while `ptrace`d. **Use `real_parent`**, since the question is about genealogy, not debugging.

- When a parent dies, children are *reparented* to a sub-reaper or to the namespace init. From that moment they are no longer in the former parent's tree. This is the correct behaviour for your syscall, but it must be documented and tested.
- PIDs are namespaced. User space passes a number valid in *its own* PID namespace: use `find_task_by_vpid()`, not `find_task_by_pid_ns(..., &init_pid_ns)`.

2.3 Resource accounting already in the kernel

You are not meant to invent accounting; read what is there.

Quantity	Where it lives	Helper
User/system CPU time (whole process)	per-thread & `signal_struct`	`thread_group_cputime_adjusted()`
Resident set size	`mm_struct` counters	`get_mm_rss(mm)`
Virtual size	`mm->total_vm` (pages)	direct read
Thread count	`signal->nr_threads`	`get_nr_threads(t)`
Page faults, ctx switches, I/O	per-task / `signal_struct`	see `k_getrusage()` in `kernel/sys.c`
Limits (soft, hard)	`signal->rlim[]`	`task_rlimit()`, `task_rlimit_max()`

`kernel/sys.c`: `k_getrusage()` is the best worked example of aggregating usage over a thread group; read it before you start.

3 Design

3.1 Interface

Modern syscall conventions (see `Documentation/process/adding-syscalls.rst`) ask for a `flags` argument and an extensible, size-versioned struct passed by pointer, as with `clone3` and `openat2`.

```
long pidmon(pid_t pid, unsigned int flags,
            struct pidmon_usage __user *uattr, size_t usize);
```

- `pid`: target TGID in the caller's PID namespace. `0` means "myself".
- `flags`: must be `0` or a defined bit. Reject unknown bits with `-EINVAL` (otherwise you can never define them later). Suggested: `PIDMON_F_NO_ENFORCE` (report only, a dry run, very useful for testing).
- `uattr`/`usize`: output buffer and its size, copied with `copy_struct_to_user()`, which handles older/newer user structs.
- Returns `0` on success; the action taken is reported *in the struct*.

Error	Meaning
`ESRCH`	no such process in the caller's namespace
`EPERM`	target exists but is not the caller or a descendant
`EINVAL`	bad flags or `usize` too small
`EFAULT`	bad user pointer

Design decision

Return `ESRCH` vs. `EPERM`? Returning `EPERM` for "exists but not yours" leaks existence of arbitrary PIDs; but `kill(2)` does the same and PIDs are already visible in `/proc`. Keep `EPERM`; it is more helpful for the grader and for debugging.

3.2 Where do the "limits" come from?

The problem passes only a `pid`, so the limits cannot be call arguments. The natural source is the target's existing `RLIMIT_*` values, set via `setrlimit(2)`/`prlimit(2)`. This needs no new storage and fits Linux semantics. Suggested resources:

Limit	Measured against	Note
`RLIMIT_CPU`	utime+stime, seconds	kernel already enforces this via timers; yours adds on-demand enforcement
`RLIMIT_AS`	`total_vm`×page size	kernel enforces at `mmap`; check anyway
`RLIMIT_RSS`	`get_mm_rss()`×page size	**not enforced by Linux at all**, so your call is the only enforcer

Design decision

Alternative: a new per-task limits table (e.g. in `signal_struct`) set by a second syscall or `prctl`. More work, more freedom, and more risk. Use `RLIMIT_*` unless the assignment demands otherwise.

3.3 What is the "appropriate action"?

Proposed policy, per resource, strongest wins:

Condition	Action
usage < soft limit (or limit is `RLIM_INFINITY`)	none
soft ≤ usage < hard	send `SIGXCPU` for CPU, `SIGTERM` otherwise (catchable warning)
usage ≥ hard	send `SIGKILL`

Report the verdict in the struct (`action` field) so the caller knows what happened.

4 Environment setup

4.1 Get the source

```
git clone https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git
cd linux
git checkout -b pidmon-syscall # work on a branch, commit often
```

Tracking Linus's tree is "latest mainline". Note the tag you started from (`git describe`) in your report.

4.2 Build tools (Arch Linux)

```
sudo pacman -S --needed base-devel bc cpio pahole libelf perl python \
    flex bison openssl qemu-system-x86 busybox
```

4.3 Never test on your host

A buggy syscall can oops or hang the machine. Use a VM. Two good options:

- **virtme-ng** (fastest loop): `vng -v -build` builds the tree and boots it in QEMU sharing your filesystem; `vng -- ./test` runs a program inside.
- **QEMU + initramfs**: build with a minimal config, pack your static test binary and busybox into an initramfs, boot with `-kernel arch/x86/boot/bzImage -initrd ... -append "console=ttyS0" -nographic`.

Your test programs must be built `-static` when using a bare initramfs.

4.4 Config

```
make defconfig # or: vng --kconfig
scripts/config -e DEBUG_KERNEL -e DEBUG_INFO_DWARF5 -e PROVE_LOCKING \
               -e KASAN -e DEBUG_ATOMIC_SLEEP -e DEBUG_LIST
make olddefconfig && make -j"$(nproc)"
```

`PROVE_LOCKING` (lockdep), `DEBUG_ATOMIC_SLEEP` ("sleeping in RCU section" bugs) and `KASAN` catch precisely the mistakes this syscall invites. They slow the kernel; that is fine in a VM.

5 Implementation steps

5.1 Step 1: the user-space ABI header

Create `include/uapi/linux/pidmon.h`. Rules for UAPI structs: fixed-width types only; explicit 64-bit alignment (`aligned_u64`) so 32-bit and 64-bit layouts agree; no pointers-sized types; add new fields only at the end; no padding holes.

```
/* SPDX-License-Identifier: GPL-2.0 WITH Linux-syscall-note */
#ifndef _UAPI_LINUX_PIDMON_H
#define _UAPI_LINUX_PIDMON_H
#include <linux/types.h>

#define PIDMON_F_NO_ENFORCE (1U << 0) /* report only, take no action */
#define PIDMON_F_VALID  (PIDMON_F_NO_ENFORCE)

#define PIDMON_ACT_NONE  0
#define PIDMON_ACT_WARNED 1 /* soft limit exceeded */
#define PIDMON_ACT_KILLED 2 /* hard limit exceeded */

struct pidmon_usage {
        __u64 utime_ns;  /* user CPU time, whole thread group */
        __u64 stime_ns;  /* system CPU time */
        __u64 rss_bytes;
        __u64 vm_bytes;
        __u32 nr_threads;
        __u32 action;  /* PIDMON_ACT_* taken (or that would be taken) */
        __u32 exceeded;  /* bitmask of RLIMIT_* numbers that tripped */
        __u32 __pad;
        /* new fields go below this line */
};
#define PIDMON_USAGE_SIZE_VER0 40 /* sizeof() at first release */
#endif
```

The header is exported to user space by `make headers_install`; `include/uapi/linux/Kbuild` usually needs no edit since headers are picked up by wildcard, but verify.

5.2 Step 2: the implementation file

Create `kernel/pidmon.c` and add `obj-y += pidmon.o` to `kernel/Makefile`. (You may instead put the code in `kernel/sys.c`; a separate file is cleaner for review.)

2a. Tree-membership check

Walk *upwards* from the target. This is cheap (O(depth)), needs no global lock, and is exactly the definition of "descendant".

```
/* Caller must hold rcu_read_lock(). */
static bool pidmon_in_my_tree(struct task_struct *t)
{
        struct task_struct *p;
```

```
        for (p = t; p != &init_task; p = rcu_dereference(p->real_parent))
                if (same_thread_group(p, current))
                        return true; /* t == me, or I am an ancestor */
        return false;
}
```

Why this is correct. The first iteration (`p == t`) handles "self" (any thread of my process counts). Every later iteration moves one generation up, so reaching a task in my thread group means I am some ancestor. The loop stops at `init_task` (the idle task, whose `real_parent` is itself), so it always terminates. Using `same_thread_group` rather than `p == current` matters because `real_parent` points to the *thread* that called `fork`, which need not be the caller's thread.

Pitfall

`real_parent` is RCU-protected; dereference it only inside `rcu_read_lock()` with `rcu_dereference`. Do *not* use `task->parent` (ptrace-modified) and do *not* compare PIDs as integers (namespaces, reuse).

2b. Gathering usage

```
static void pidmon_collect(struct task_struct *t, struct pidmon_usage *u)
{
        struct mm_struct *mm;
        u64 ut, st;

        thread_group_cputime_adjusted(t, &ut, &st); /* may take locks; fine */
        u->utime_ns = ut;
        u->stime_ns = st;
        u->nr_threads = get_nr_threads(t);

        mm = get_task_mm(t);    /* NULL for zombies/kthreads */
        if (mm) {
                u->rss_bytes = (u64)get_mm_rss(mm) << PAGE_SHIFT;
                u->vm_bytes = (u64)mm->total_vm << PAGE_SHIFT;
                mmput(mm);    /* may sleep: not under RCU */
        }
}
```

Zombie targets have no `mm`; reporting zero memory is correct. Add more fields (page faults, context switches) by following `k_getrusage()`.

2c. Deciding the action

```
static u32 pidmon_judge(struct task_struct *t, const struct pidmon_usage *u,
                        u32 *exceeded, int *sig)
{
        struct { int res; u64 val; int soft_sig; } chk[] = {
                { RLIMIT_CPU, (u->utime_ns + u->stime_ns) / NSEC_PER_SEC, SIGXCPU },
                { RLIMIT_AS, u->vm_bytes, SIGTERM },
                { RLIMIT_RSS, u->rss_bytes, SIGTERM },
        };
        u32 act = PIDMON_ACT_NONE;
        int i;

        *exceeded = 0; *sig = 0;
        for (i = 0; i < ARRAY_SIZE(chk); i++) {
                unsigned long soft = task_rlimit(t, chk[i].res);
                unsigned long hard = task_rlimit_max(t, chk[i].res);
```

```
                if (hard != RLIM_INFINITY && chk[i].val >= hard) {
                        *exceeded |= BIT(chk[i].res);
                        act = PIDMON_ACT_KILLED;
                        *sig = SIGKILL;
                } else if (soft != RLIM_INFINITY && chk[i].val >= soft) {
                        *exceeded |= BIT(chk[i].res);
                        if (act < PIDMON_ACT_WARNED) {
                                act = PIDMON_ACT_WARNED;
                                *sig = chk[i].soft_sig;
                        }
                }
        }
        return act;
}
```

Why `task_rlimit(t,...)` and not `rlimit(...)`: the latter reads *current's* limits; you want the *target's*. Mind the units: `RLIMIT_CPU` is in seconds, memory limits in bytes. Compare in the same unit.

2d. The syscall itself

```
SYSCALL_DEFINE4(pidmon, pid_t, pid, unsigned int, flags,
                struct pidmon_usage __user *, uattr, size_t, usize)
{
        struct pidmon_usage u = {};
        struct task_struct *t;
        struct pid *spid;
        int sig, ret;

        if (flags & ~PIDMON_F_VALID)
                return -EINVAL;
        if (usize < PIDMON_USAGE_SIZE_VER0)
                return -EINVAL;
        if (pid < 0)
                return -EINVAL;

        rcu_read_lock();
        t = pid ? find_task_by_vpid(pid) : current;
        if (!t) {
                rcu_read_unlock();
                return -ESRCH;
        }
        t = t->group_leader;   /* operate on the process */
        if (!pidmon_in_my_tree(t)) {
                rcu_read_unlock();
                return -EPERM;
        }
        get_task_struct(t);   /* pin it past the RCU section */
        spid = get_pid(task_tgid(t));  /* pin the pid: no reuse races */
        rcu_read_unlock();

        pidmon_collect(t, &u);
        u.action = pidmon_judge(t, &u, &u.exceeded, &sig);

        ret = copy_struct_to_user(uattr, usize, &u, sizeof(u), NULL);
        if (!ret && sig && !(flags & PIDMON_F_NO_ENFORCE))
                kill_pid(spid, sig, 1);  /* 1 = SEND_SIG_PRIV */

        put_pid(spid);
        put_task_struct(t);
        return ret;
}
```

Walk-through of the choices.

- *Validate before touching anything*: flags, size, sign of pid.
- *RCU section covers lookup + tree check only*: you may not sleep inside it. `get_task_mm`/`mmput`, `copy_to_user` and `kill_pid` are all done outside, on pinned objects.
- *`get_pid` before `kill`*: between your check and the signal the target could exit and its numeric PID be recycled. Signalling a `struct pid *` you hold a reference on can only reach the process you vetted (or nothing).
- *Copy out before enforcing*: if the user pointer is bad you fail with `EFAULT` without having killed anything.
- *Authorisation*: being an ancestor *is* the permission, so `kill_pid(..., priv=1)` deliberately skips the usual uid/capability check (a child may have changed uid; a parent still has authority here). State this explicitly in your report. If you prefer conservative semantics, pass `priv=0` and `kill(2)` rules apply.
- *Self-enforcement*: if `pid` is yourself and you are over a hard limit, you will be killed. That is the specified behaviour.

Pitfall

Do not take `tasklist_lock` unless you walk *down* the child lists. The upward walk under RCU avoids it. Do not call anything that may sleep between `rcu_read_lock()` and `rcu_read_unlock()`; `DEBUG_ATOMIC_SLEEP` will flag it.

2e. Includes

At the top of `kernel/pidmon.c`:

```
// SPDX-License-Identifier: GPL-2.0
#include <linux/syscalls.h>
#include <linux/sched.h>
#include <linux/sched/signal.h>
#include <linux/sched/cputime.h>
#include <linux/sched/mm.h>
#include <linux/sched/task.h>
#include <linux/mm.h>
#include <linux/pid.h>
#include <linux/uaccess.h>
#include <linux/resource.h>
#include <uapi/linux/pidmon.h>
```

5.3 Step 3: register the syscall

1. **Prototype** in `include/linux/syscalls.h`:

   ```
   struct pidmon_usage;
   asmlinkage long sys_pidmon(pid_t pid, unsigned int flags,
                              struct pidmon_usage __user *uattr, size_t usize);
   ```

2. **x86-64 table**, `arch/x86/entry/syscalls/syscall_64.tbl`: add a line in the `common` range using the next unused number (open the file, scroll to the end of the `common` block before the `x32` section starting at 512, and take the successor of the highest number):

   ```
   <NR> common pidmon  sys_pidmon
   ```

3. **x86-32 table** (optional but good practice), `arch/x86/entry/syscalls/syscall_32.tbl`: same number, `i386` ABI, entry `sys_pidmon`. Because your struct uses only fixed-width types, no compat wrapper is needed.

4. **Other architectures** (arm64, riscv, ...) use the generic list: `include/uapi/asm-generic/unistd.h` (and, in recent trees, `scripts/syscall.tbl`). The assignment targets x86, so say in your report that you only wired x86, or do the generic one too; `scripts/checksyscalls.sh` will warn about tables left inconsistent.

Why numbers are append-only: syscall numbers are ABI. Never reuse or renumber.

5.4 Step 4: build

```
make -j"$(nproc)" 2>&1 | tee build.log
```

Typical first-build errors: missing include for `thread_group_cputime_adjusted` (`linux/sched/cputime.h`), an undeclared `RLIM_INFINITY` (`linux/resource.h`), and a mismatch between the table's entry name and the `SYSCALL_DEFINE` name.

6 Testing

6.1 User-space wrapper

Until glibc knows your syscall, use `syscall(2)`:

```c
#define _GNU_SOURCE
#include <unistd.h>
#include <sys/syscall.h>
#include <linux/pidmon.h>  /* from make headers_install */
#define __NR_pidmon <NR>

static long pidmon(pid_t pid, unsigned flags, struct pidmon_usage *u)
{
       return syscall(__NR_pidmon, pid, flags, u, sizeof(*u));
}
```

6.2 Test matrix

Write one C program with these cases; each should print PASS/FAIL.

Scenario	Expected	Verifies
`pid = getpid()` and `pid = 0`	0, sane numbers	R3 self
direct child	0	child
grandchild, great-grandchild (fork chains)	0	deep tree
parent (`getppid()`)	`EPERM`	upward is forbidden
sibling	`EPERM`	not a descendant
`pid = 1`, random live pid	`EPERM`	unrelated
nonexistent pid	`ESRCH`	lookup failure
negative pid, bad flags, small `usize`	`EINVAL`	validation
`uattr = NULL` / bad pointer	`EFAULT`	no kill happened
child busy-loops, `RLIMIT_CPU` hard=1s	after ∼1 s call: `KILLED`, child dies by `SIGKILL`	enforcement CPU
child `malloc`+touch, `RLIMIT_RSS` soft below usage	`WARNED`, child gets `SIGTERM`	enforcement soft
same with `PIDMON_F_NO_ENFORCE`	action reported, child still alive	dry run
orphan: middle process exits, grandchild reparented	`EPERM` for former grandparent	reparenting
multithreaded child	`nr_threads`, CPU summed over threads	thread group
zombie child (not yet `wait`ed)	0, zero memory	`mm == NULL` path
inside `unshare -p` namespace	vpid numbering works	namespaces

Set limits in the *child* with `setrlimit` (or the parent with `prlimit(child, ...)`) before the call. Note that you cannot raise a hard limit back without privilege; design tests accordingly.

6.3 Stress / debug

- Run the fork-chain test in a loop of thousands of iterations with random exits to shake out reference-count and RCU bugs.
- Run under the debug config of Section 3.3 and check `dmesg` for lockdep, KASAN or "sleeping function called from invalid context" reports. A clean `dmesg` is part of your evidence.
- Use `strace` inside the VM to confirm arguments; use `printk`/`trace_printk` sparingly and remove it afterwards.

7 Polish expected of a mainline-quality change

- `scripts/checkpatch.pl -strict` on your patch; fix style (tabs, 80–100 columns, SPDX line).
- A selftest under `tools/testing/selftests/pidmon/` (Makefile + test using the `kselftest_harness.h` `TEST()` macros). Your matrix above is the content.
- Documentation: `Documentation/userspace-api/pidmon.rst` describing semantics, errors and the tree rule.
- Mirror the syscall number into the tools copies if your tree has them (`tools/include/uapi/asm-generic/unist` `tools/perf/arch/x86/entry/syscalls/syscall_64.tbl`); `tools/perf/check-headers.sh` complains otherwise.
- Commit with a proper message: imperative subject, `Signed-off-by:` line (`git commit -s`), a body explaining *why*. Produce the patch with `git format-patch -1`. Do *not* actually mail it to LKML for a course assignment.

8 Verification greps (names that drift between releases)

Run these in the tree before trusting any signature in this document.

```
grep -rn "thread_group_cputime_adjusted" include/linux/sched/cputime.h
grep -n "task_rlimit\|task_rlimit_max" include/linux/sched/signal.h
grep -n "copy_struct_to_user" include/linux/uaccess.h
grep -n "int kill_pid(" kernel/signal.c include/linux/signal.h
grep -n "get_mm_rss\|get_task_mm" include/linux/mm.h include/linux/sched/mm.h
grep -n "same_thread_group\|get_nr_threads" include/linux/sched/signal.h
tail -n 30 arch/x86/entry/syscalls/syscall_64.tbl # next free number
sed -n '/k_getrusage/,/^}/p' kernel/sys.c # how real accounting is aggregated
```

Also read, in order: `Documentation/process/adding-syscalls.rst`, `kernel/sys.c` (`getrusage`, `prlimit`), `kernel/signal.c` (`kill_pid_info`), and one recent small syscall addition via `git log -oneline - arch/x86/entry/syscalls/syscall_64.tbl`; open the top commit with `git show` to see exactly which files such a change touches.

9 Suggested work plan

1. Clone, build the *unmodified* kernel, boot it in a VM, run a trivial program. (Confirms the toolchain.)
2. Add a stub syscall that just returns 42; wire the table, call it from user space. (Confirms plumbing.)
3. Implement tree check + the `EPERM`/`ESRCH` logic; test the relationship rows.
4. Add `pidmon_collect`; compare against `/proc/<pid>/stat` and `status`.
5. Add `pidmon_judge` with `NO_ENFORCE`; then enable signals.
6. Debug-config soak test, selftest, documentation, report.

Open questions for your instructor

1. Which resources must be reported (is CPU + RSS + VM enough)?
2. Are limits meant to be the target's `RLIMIT_*`, or a new mechanism?
3. Is "appropriate action" terminate-on-hard-limit with warning-on-soft, or something else (e.g. throttle, `SIGSTOP`)?
4. Should the syscall be restricted to x86-64, and what is the exact required name/signature?